

KAN-IDS — consolidamento sperimentale

Protocollo leakage-free, validazione 5-fold × 3 seed, confronto con baseline identiche e cross-domain
TON_IoT ↔ BoT-IoT

Emanuele Pio De Bernardis — agosto 2026 — github.com/emanuelepiodebernardis/kan-ids-v2

1. Protocollo

Ogni trasformazione che apprende dai dati — ranking per mutual information, vocabolari delle categoriche, quantili — è ora fittata **esclusivamente sul training** di ciascun fold. Nella versione precedente il ranking era calcolato su un campione dell'intero dataset prima dello split, e i vocabolari categorici su train+test. La correzione vive in un solo modulo (*kanids/preprocessing.py*) usato da tutti gli script; due test automatici impediscono che il difetto rientri.

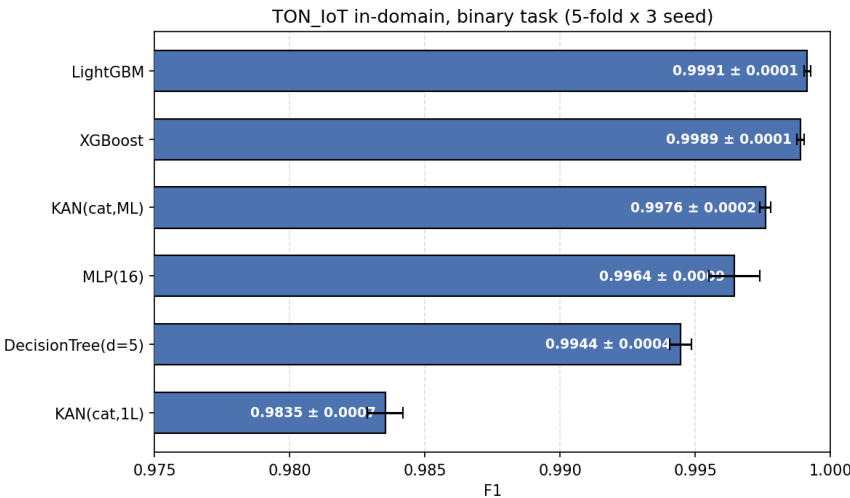
Effetto misurato del difetto: nullo. La selezione per-fold sceglie le stesse 10 feature numeriche in 15 fold su 15, e in-domain non esiste alcuna categoria assente dal training. I numeri precedentemente riportati restano quindi validi; il protocollo andava comunque corretto, perché la sua validità non può dipendere da quanto grande sia risultato l'effetto.

Validazione: **StratifiedKFold(5) ripetuta su 3 seed = 15 fit per modello**, media ± deviazione standard. La stratificazione usa sempre l'etichetta a 10 classi, così i modelli binari e multiclass vedono fold identici e la classe rara (MITM, 0,49% dei flussi) è presente ovunque.

2. Tabella comparativa — TON_IoT in-domain

Modello	F1 binario	PR-AUC	FPR	Macro-F1 10 classi	F1 MITM
LightGBM	0.9991 ± 0.0001	1.0000	0.0023	0.9680 ± 0.0021	0.767
XGBoost	0.9989 ± 0.0001	1.0000	0.0041	0.9666 ± 0.0021	0.761
KAN(cat,ML)	0.9976 ± 0.0002	0.9999	0.0037	0.9374 ± 0.0036	0.541
MLP(16)	0.9964 ± 0.0009	0.9998	0.0088	0.9182 ± 0.0107	0.386
DecisionTree(d=5)	0.9944 ± 0.0004	0.9981	0.0075	0.7633 ± 0.0033	0.151
KAN(cat,1L)	0.9835 ± 0.0007	0.9985	0.0208	0.8767 ± 0.0014	0.270

15 fit per modello e per task. Spazio a 14 feature (10 numeriche + 4 categoriche); tutti i modelli ricevono esattamente lo stesso input.



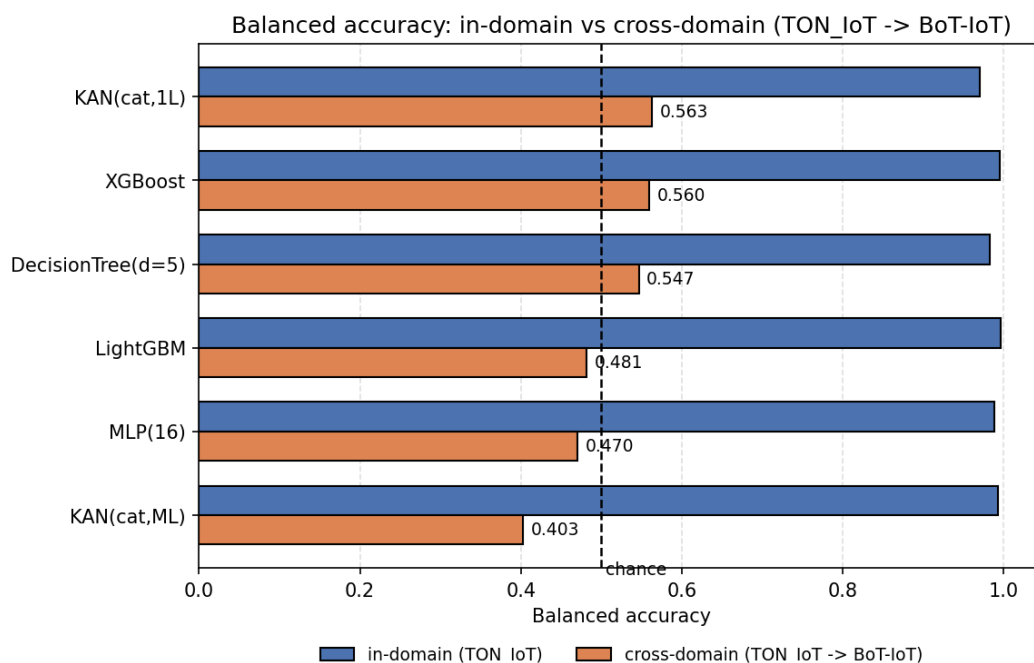
3. Cross-domain TON_IoT ↔ BoT-IoT

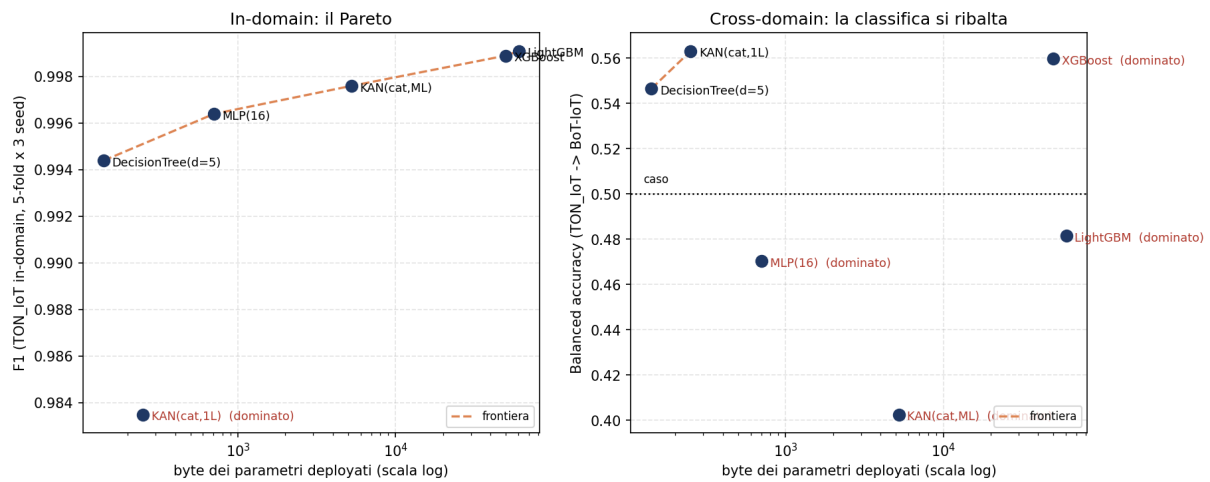
Task binario su uno **spazio armonizzato a 13 feature candidate**, calcolate con la stessa formula sui due dataset (durata, byte e pacchetti IP per direzione, totali, asimmetrie, payload medi, rate) più protocollo e stato mappati su un alfabeto semantico comune. Escluse porte e indirizzi (identificatori di testbed), gli aggregati a finestra di BoT-IoT e i metadati DNS/SSL/HTTP di TON_IoT. Nel cross-domain il target entra **solo** nella valutazione.

Nota sulle metriche. BoT-IoT è attacco al 99,987%: in quel regime la PR-AUC sulla classe positiva vale ~1 per costruzione e non discrimina (nei run TON→BoT è 0,9999 mentre i modelli sono al caso). Si riportano i due recall e la loro media, la balanced accuracy, dove **0,50 = caso**.

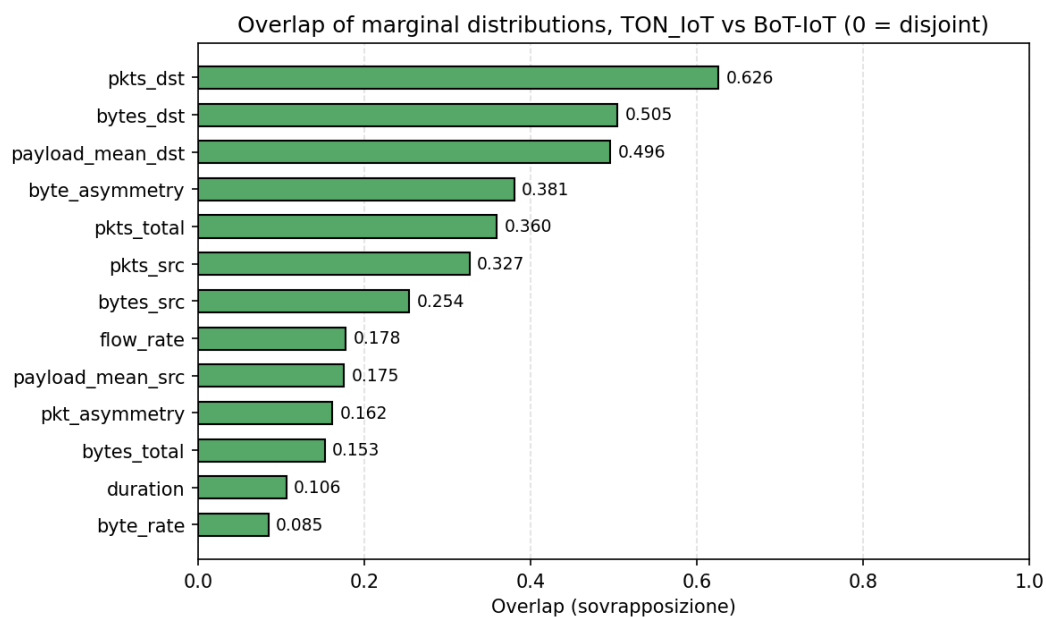
Modello	TON in-dom.	TON→BoT	δ	BoT in-dom.	BoT→TON	δ
MLP(16)	0.9885	0.4703	0.5182	0.9460	0.7343	0.2117
LightGBM	0.9962	0.4815	0.5148	0.9966	0.7171	0.2795
XGBoost	0.9948	0.5597	0.4352	0.9769	0.6508	0.3261
KAN(cat,ML)	0.9933	0.4026	0.5907	0.9979	0.6581	0.3398
KAN(cat,1L)	0.9700	0.5632	0.4068	0.9935	0.5989	0.3946
DecisionTree(d=5)	0.9828	0.5466	0.4362	0.9953	0.4651	0.5301

Balanced accuracy. In-domain: 15 fit. Direzioni cross: 3 seed, training sull'intero source, valutazione sull'intero target.



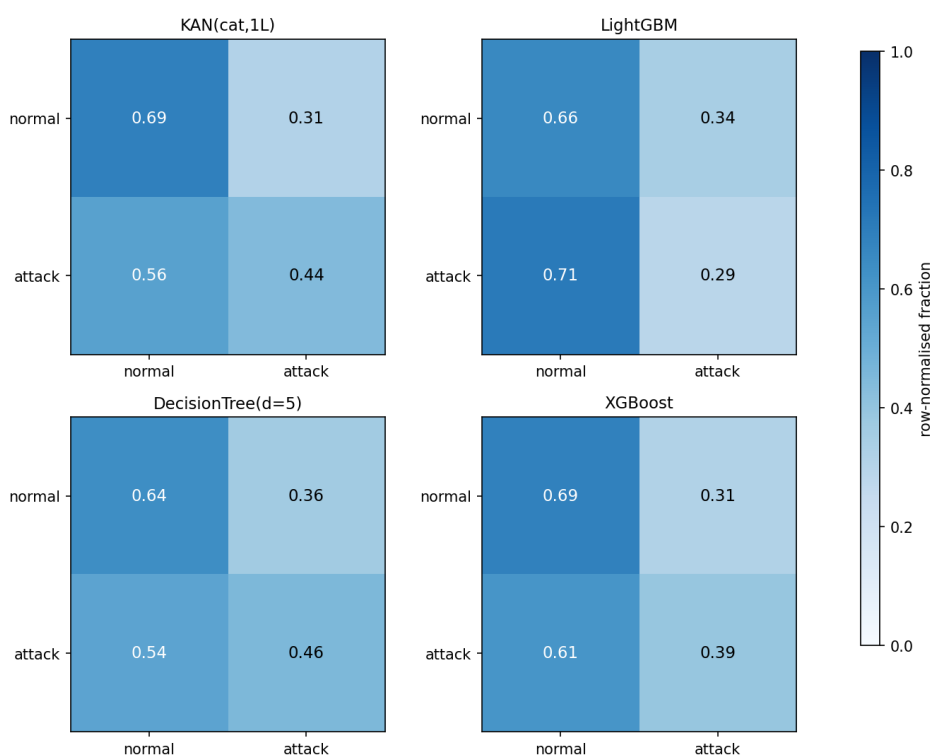


Frontiera di Pareto dimensione/accuratezza. A sinistra in-domain: il modello KAN da 250 byte è dominato dall'albero profondo 5, che occupa 141 byte ed è più accurato. A destra cross-domain: la classifica si ribalta e il single-layer è il modello che trasferisce meglio.



Sovrapposizione delle distribuzioni marginali fra i due domini. Valori sotto 0,2 significano supporti quasi disgiunti.

Confusion matrices, TON_IoT -> BoT-IoT (row-normalised)



Matrici di confusione TON_IoT → BoT-IoT, normalizzate per riga. LightGBM classifica come normale il 71% degli attacchi di BoT-IoT.

4. Analisi critica

4.1 Il confronto con le baseline era viziato

La versione precedente dichiarava che il modello da 246 byte supera gli ensemble ad alberi «sullo stesso spazio di feature deployabile». Non è così: quel confronto metteva la KAN sullo spazio grezzo a 14 feature e le baseline sullo spazio derivato a 10 feature del paper. Con input identici, **LightGBM raggiunge 0,9991 contro 0,9835 della KAN single-layer**, vincendo 15 fold su 15 (t-test $p = 3,7 \cdot 10^{-4}$). Il claim è stato corretto nel README.

4.2 La profondità colma il divario, e spiega perché esisteva

La KAN single-layer è un modello additivo generalizzato: somma di funzioni univariate, incapace per costruzione di rappresentare interazioni fra feature. La multi-layer, che può, guadagna **+0,0141 di F1 vincendo 15 fold su 15** e arriva a $0,9976 \pm 0,0002$. È la misura diretta che il divario riguardava le interazioni, non la capacità né l'ottimizzazione — e trasforma un punto debole in un risultato architetturale.

4.3 A parità di conteggio, il modello da 250 byte è dominato

Contando ogni modello con la stessa regola — byte dei parametri memorizzati in rappresentazione table-driven — l'albero di profondità 5 occupa **141 byte** con F1 **0,9944**: è insieme **più piccolo e più accurato** del modello KAN single-layer (250 byte, 0,9835), che risulta quindi **dominato** sulla frontiera di Pareto in-domain. L'albero è anche invariante a trasformazioni monotone, quindi non richiede alcun preprocessing, mentre la catena integer end-to-end della KAN costa 822 byte comprese le tabelle. «Accuratezza per byte» non è un argomento difendibile per il single-layer su TON_IoT, e il repository non lo sostiene più.

Tre cose invece reggono, e sono quelle su cui costruire. **(a)** La KAN multi-layer sta sulla frontiera: 5,2 KB e F1 0,9976 contro l'MLP TensorFlow Lite Micro del paper originale, 13 KB e 0,9959 con 95 feature — più piccola, più accurata, 14 feature invece di 95. **(b)** Nel cross-domain la classifica si ribalta: il single-layer è il modello che trasferisce meglio (0,5632) mentre l'albero scende a 0,5466 ed è il peggiore di tutti nella direzione BoT→TON (0,4651). **(c)** La KAN offre ciò che un albero non ha: calibrazione conformal sul modello intero deployato, forma simbolica chiusa e tabelle riscrivibili, che sono il presupposto della ricalibrazione on-device.

4.4 Il cross-domain non degrada: collassa

TON→BoT lascia ogni modello fra 0,40 e 0,56 di balanced accuracy, cioè al caso o sotto. Il δ quantificato nel paper era $\leq 5,95$ punti; qui siamo a **40–52 punti**. Due osservazioni non ovvie: il compromesso fra capacità e trasferibilità si misura **dentro la stessa famiglia** — aggiungere profondità alla KAN vale +0,0141 di F1 sul binario e +0,0608 di macro-F1 sulle 10 classi, e le costa il transfer per intero (0,4026 di balanced accuracy, **sotto il caso**, il peggiore di tutti i modelli), mentre la single-layer, ultima in-domain, è prima cross-domain; e la direzione BoT→TON non è degradata ma **indeterminata** — con 477 flussi normali in training la varianza fra seed supera la differenza fra modelli, quindi qualunque classifica in quella direzione sarebbe rumore.

La causa è visibile prima di addestrare qualsiasi cosa: le marginali non si sovrappongono (byte_rate 0,085, duration 0,106). TON_IoT ha flussi brevi e bidirezionali, il 5% di BoT-IoT è dominato da flood UDP lunghi e unidirezionali. Il 21,3% dei flussi di TON_IoT porta uno stato di connessione mai visto addestrando su BoT-IoT. Gli edge categorici armonizzati sono ciò che tiene il transfer sopra il caso: rimuoverli costa 0,08–0,16 di balanced accuracy.

4.5 La catena integer end-to-end ora è in C, e ha rivelato due bug

Il percorso dai contatori grezzi alla decisione gira interamente in aritmetica intera nel firmware: **200 golden vector su 200 con logit bit-identico** al riferimento Python, e l'ispezione dell'assembly del percorso di

inferenza mostra **zero istruzioni in virgola mobile**. Il percorso end-to-end precedente (*mcu_e2e*) interpolava 10.000 knot del QuantileTransformer in doppia precisione: end-to-end nella struttura, ma non integer-only nel runtime. Il porting ha fatto emergere tre difetti che si sarebbero manifestati solo sul dispositivo: il moltiplicatore di scala della feature con scala massima quantizza a esattamente 32768, che in *int16* va in overflow silenzioso; la divisione intera di Python arrotonda verso $-\infty$ mentre quella del C tronca verso zero, differenza osservabile sulle due feature di asimmetria, che hanno numeratore di segno qualsiasi; e la stessa saturazione a 2^{15} ricompare nella LUT della tangente iperbolica del modello a 10 classi.

Anche la **catena a 10 classi** e' ora in C: contatori grezzi $\rightarrow$ ricerca binaria sulle soglie per-feature $\rightarrow$ z in Q12 $\rightarrow$ primo strato di spline int8 con tabelle categoriche $\rightarrow$ LUT tanh $\rightarrow$ secondo strato $\rightarrow$ argmax, tutto intero. **200 golden vector su 200 con tutti e dieci gli accumulatori bit-identici** al riferimento, argmax identico, macro-F1 0,9352 contro 0,9378 della pipeline float (agreement 99,42%), 13,6 KB di tabelle. Le soglie restano a 64 bit perche' su *TON_IoT src_bytes* e *dst_bytes* arrivano a $3,9 \cdot 10^9$: sono i contatori di byte a imporre i 64 bit, non la durata.

4.6 Sul multiclass la profondità pesa quattro volte di più

Con lo stesso protocollo sulle 10 classi la KAN multi-layer fa **$0,9374 \pm 0,0036$** contro 0,8767 della single-layer: **+0,0608 di macro-F1, 15 fold su 15**, contro il +0,0141 del binario. Separare le famiglie di attacco richiede interazioni fra feature molto più di quanto ne richieda separare attacco da normale — stesso argomento strutturale, effetto quattro volte più grande. E l'albero profondo 5, che sul binario era il modello più piccolo e più accurato, qui crolla all'ultimo posto, 0,1741 sotto la multi-layer: il suo dominio in-domain era specifico del problema binario e non sopravvive al task più difficile.

4.7 Il soffitto su MITM è informativo, e riguarda tutti

Sul multiclass la classe MITM (1.043 flussi, 0,49%) è il collo di bottiglia per **ogni** modello: LightGBM 0,767, MLP 0,386, KAN single-layer 0,270, albero profondo 5 0,151. Tutte le altre classi stanno sopra 0,88. Poiché anche il modello più capace si ferma a 0,77, il limite è nell'informazione disponibile nello spazio di feature, non nell'architettura — coerente con il fallimento già documentato di focal loss, SMOTENC e class weighting.

5. Stato e passi successivi

Punto	Stato
1. Protocollo leakage-free	completato, con misura dell'effetto
2. Multi-layer con lo stesso protocollo	completato: $0,9976 \pm 0,0002$
3. BoT-IoT, 4 direzioni	completato, con analisi del degrado
4. Baseline identiche	completato su binario e multiclass
6. Riproducibilità da clone pulito	completato, verificato
5. Integer-only end-to-end (binario)	completato: 200/200 bit-esatti, 822 B
5. Integer-only end-to-end (10 classi)	completato: 200/200 bit-esatti, 13,6 KB
Ingombro dei parametri (asse dimensione)	misurato: results/footprint.csv
models/ + manifest (protocollo, seed, metriche)	completato
Conformal e forma simbolica sotto v2	rigenerati: coperture 99,05/94,90/90,23
Benchmark fisici (latenza, energia, code size)	da fare — richiedono le board
Focal loss e SMOTENC (risultati negativi)	ancora v1; corroborati indipendentemente in v2
Multi-layer multiclass in CV 5x3	completato: $0,9374 \pm 0,0036$ su 15 fit
CIC-IoT-2023 come terzo dataset	non iniziato (obiettivo secondario)

La priorità che suggerirei è il benchmark fisico: senza gli ingombri misurati, il confronto con l'albero profondo 5 resta aperto ed è la prima obiezione che farebbe un revisore. Il cross-domain, dal canto suo, fornisce la motivazione quantificata al passo successivo: a 40 punti di gap l'adattamento on-device non è un miglioramento marginale, ma la condizione perché un IDS embedded funzioni fuori dal laboratorio in cui è stato addestrato.